

Energy-Aware Flexible Job-Shop Scheduling via a Memetic NSGA-II

Adeleke Olorunnisola Oyeyemi

August 2026

Abstract

This document describes the design, implementation, and results of a bi-objective optimisation mini project that schedules the Flexible Job-Shop Scheduling Problem (FJSP) to jointly minimise *makespan* and *energy cost*, where energy cost is computed from a real, time-varying electricity price series rather than an assumed flat rate. The scheduler is a *memetic NSGA-II*: the multi-objective genetic algorithm NSGA-II [6], hybridised with a greedy local-search refinement step, and solving real Branimart [1993] FJSP benchmark instances against real ENTSO-E day-ahead electricity prices (Belgium, February 2022). The project is methodologically inspired by Burmeister et al. [4], who introduce and validate a memetic NSGA-II for FJSP under real-time energy tariffs. This document covers the motivation, problem formulation, algorithm design, data sources, implementation, and results, and closes with an explicit comparison of what is simplified relative to the source paper. Source code and full results are hosted at <https://github.com/Olorunnisola01/energy-aware-fjsp-nsga2>.

Contents

1	Introduction and Motivation	3
2	Related Work	3
2.1	Energy-Cost-Aware Scheduling	3
2.2	NSGA-II and Memetic Algorithms	3
2.3	Benchmark Instances	4
3	Problem Formulation	4
3.1	Inputs	4
3.2	Decision Variables	4
3.3	Objectives	4
4	Methodology	4
4.1	Solution Encoding	4
4.2	Memetic NSGA-II	5
4.3	Duplicate Removal	5
5	Data	5
5.1	Benchmark Instances	5
5.2	Electricity Price Data	6
6	Implementation	6
7	Experimental Setup	6

8	Results	7
8.1	Convergence	7
8.2	Pareto Front	7
8.3	Schedule (Gantt Chart)	9
9	Discussion: Relationship to Burmeister et al. (2024)	9
10	Limitations and Future Work	10
11	Conclusion	10

1 Introduction and Motivation

Electricity grids are incorporating increasing shares of intermittent renewable generation (wind, solar), which introduces volatility into energy supply and, consequently, into energy prices [1, 9]. Under real-time pricing (RTP) tariffs, the cost of electricity can change at hourly (or finer) intervals depending on current market conditions. This creates an opportunity, and an incentive, for energy-flexible manufacturers to practice *demand response*: shifting production toward hours when energy is cheap, without any new capital investment [7].

Job-shop scheduling problems are a natural setting for this idea. In the *Flexible Job-Shop Scheduling Problem* (FJSP), each job consists of a sequence of operations, and each operation may be processed on any machine from a compatible subset, as opposed to the classical job-shop problem, where each operation is tied to exactly one machine. This flexibility gives a scheduler room to choose *when* and *on which machine* to run an operation, which is exactly the freedom needed to react to a time-varying energy price signal.

Minimising energy cost in isolation is not useful: a scheduler that always waits for the cheapest possible price window will produce arbitrarily long schedules. The problem is therefore inherently *bi-objective*, simultaneously minimising makespan (total completion time) and energy cost, and the interesting output is not a single “best” schedule but a *Pareto front* of trade-offs between the two, from which a decision-maker can pick according to their own priorities.

This project implements exactly that: a memetic NSGA-II that approximates the makespan/energy-cost Pareto front for FJSP instances under a real RTP electricity price signal.

2 Related Work

2.1 Energy-Cost-Aware Scheduling

The literature on energy-cost-aware scheduling is organised around three tariff structures [4]:

1. **Fixed rate**: a constant price regardless of time of consumption (e.g. Carlucci et al. [5], Kemmoe et al. [11]).
2. **Time-of-use (TOU) rate**: the day is divided into a small number of coarse-grained periods (e.g. off-peak / mid-peak / on-peak), each with its own price (e.g. Biel et al. [2], Zhang et al. [15]).
3. **Real-time pricing (RTP)**: the price is based on the current market value of energy and can change at hourly or finer intervals (e.g. Gong et al. [10], Shrouf et al. [14]).

Burmeister et al. [4] identify a gap in this literature: prior RTP-based work is largely restricted to single-machine or flow-shop settings and single-objective (energy-cost-only) formulations, or neglects makespan optimisation entirely. Their paper closes this gap with a *multi-objective*, RTP-aware FJSP formulation solved by a memetic NSGA-II, validated computationally against an exact Gurobi solver on the Brandimarte [1993] benchmark suite.

2.2 NSGA-II and Memetic Algorithms

NSGA-II [6] is a widely used multi-objective evolutionary algorithm. It maintains a population of candidate solutions, ranks them into non-dominated fronts, and uses a crowding-distance measure to preserve diversity along each front, producing, generation over generation, an approximation of the true Pareto front rather than a single solution.

A *memetic algorithm* [12] augments a population-based metaheuristic with a local-search refinement step applied to individuals (typically offspring), combining the global exploration of an evolutionary algorithm with the fast local exploitation of a hill-climbing search. Burmeister et al. [4] apply this idea to NSGA-II for FJSP; this project follows the same combination.

2.3 Benchmark Instances

Brandimarte [3] introduced a set of fifteen FJSP benchmark instances (mk01–mk15), commonly used in the FJSP literature (and by Burmeister et al. [4]) as a standard testbed. This project uses ten of these instances (mk01–mk10), parsed from the standard Hurink `.fjs` text format.

3 Problem Formulation

3.1 Inputs

- A set of jobs $\mathcal{J} = \{1, \dots, n\}$, each consisting of an ordered sequence of operations that must be processed in sequence.
- A set of machines $\mathcal{M} = \{1, \dots, m\}$.
- For each operation, a subset of machines that can process it, and a processing time for each such machine (processing time may differ by machine).
- A real-valued electricity price series $p(t)$ (EUR/kWh), giving the price of energy at every minute t of the planning horizon.

3.2 Decision Variables

For every operation:

1. **Machine assignment:** which eligible machine processes the operation.
2. **Sequencing position:** the relative order in which operations are dispatched, subject to (a) machine availability (a machine processes one operation at a time) and (b) job precedence (an operation cannot start before the previous operation of the same job finishes).

3.3 Objectives

$$\min f_1 = \text{makespan} = \max_i (\text{completion time of operation } i) \quad (1)$$

$$\min f_2 = \text{energy cost} = \sum_i \int_{\text{start}_i}^{\text{end}_i} p(t) dt \quad (2)$$

where the integral for each operation i is evaluated exactly by summing the per-minute price over that operation’s start/end window. No feasible solution can simultaneously minimise both objectives (a faster schedule generally cannot always wait for the cheapest price window), so the output is a Pareto-optimal set of trade-offs rather than one optimum.

4 Methodology

4.1 Solution Encoding

A candidate solution is encoded as a vector of length $2 \times n_{\text{ops}}$:

- The first half assigns, for each operation, an integer machine-choice index, taken modulo the number of machines eligible for that specific operation (so any integer is a valid, decodable gene, which avoids infeasible machine assignments by construction).
- The second half assigns each operation a sort key; operations are dispatched in ascending order of this key, subject to machine and job-precedence availability (a standard permutation-based scheduling decoder).

4.2 Memetic NSGA-II

The algorithm is standard NSGA-II (non-dominated sorting, crowding distance, tournament selection, simulated-binary crossover and polynomial mutation for integer variables, via the `pymoo` library) with one addition: after each generation produces its offspring, every offspring is refined by a *greedy local search* before being merged back into the population.

The local search alternates between two neighbourhood moves until neither improves the current solution:

1. **Machine re-assignment:** for each operation, try the next eligible machine; keep the move if it Pareto-dominates the current solution.
2. **Sequencing swap:** for each pair of operations, try swapping their sequencing keys; keep the move if it Pareto-dominates the current solution.

This hybridises NSGA-II’s global, population-level search with fast, solution-level exploitation, which is the core idea of a memetic algorithm [12] and the central methodological device in Burmeister et al. [4].

4.3 Duplicate Removal

NSGA-II’s population can end a run containing multiple genotypes that decode to the same schedule (and therefore the same objective values). The final reported Pareto front is deduplicated (`numpy.unique` on objective values) so it reads as a clean, strictly non-dominated set rather than a front inflated with repeated points.

5 Data

5.1 Benchmark Instances

Ten Brandimarte [1993] FJSP benchmark instances are used, mk01–mk10, parsed from the standard Hurink `.fjs` text format (machine indices 0-based; each job line lists its operation count, then for each operation its number of eligible machines followed by machine/time pairs). Instance files were sourced from the `SchedulingLab/fjsp-instances` public repository [13] and independently validated (every token in every file was checked to parse into a complete, internally consistent operation list with no leftover or missing tokens) before use.

Table 1: Brandimarte (1993) FJSP benchmark instances used in this project.

Instance	Jobs	Machines
mk01	10	6
mk02	10	6
mk03	15	8
mk04	15	8
mk05	15	4
mk06	10	15
mk07	20	5
mk08	20	10
mk09	20	10
mk10	20	15

The results reported in this document use `mk01` (10 jobs, 6 machines, 55 operations).

5.2 Electricity Price Data

Real day-ahead electricity prices for the Belgian bidding zone, February 2022, were sourced from the ENTSO-E Transparency Platform [8]. The raw export (EUR/MWh, timestamps like “Feb 1, 2022 12:00 AM”) is preprocessed by `parsing_data.py` into a clean (`timestamp`, `price_eur_per_kWh`) series, which is then resampled to per-minute resolution at run time so that energy cost can be integrated exactly over each operation’s start/end window rather than approximated at hourly granularity.

6 Implementation

The project is implemented in Python 3, using:

- `pymoo` for the NSGA-II algorithm framework;
- `numpy/pandas` for numerical computation and price-series handling;
- `matplotlib` for all figures.

Table 2: Key files in the project repository.

File	Purpose
<code>nsga2_fjsp_energy.py</code>	Problem definition, memetic NSGA-II, plotting, CLI
<code>parse_fjs_instance</code> (in the same file)	Hurink <code>.fjs</code> benchmark parser
<code>parsing_data.py</code>	One-off preprocessing of the raw ENTSO-E export
<code>benchmarks/brandimarte/mk01.txt–mk10.txt</code>	Benchmark instance files
<code>belgium_prices_feb2022.csv</code>	Cleaned electricity price series

Both the benchmark instance choice and the NSGA-II run are fully specified by command-line arguments and a random seed, so a given configuration reproduces the same Pareto front:

```
python nsga2_fjsp_energy.py --instance mk01 --n-gen 800 --pop-size 20 --seed 42
```

A `-random` flag is retained for generating synthetic FJSP instances of arbitrary size, independent of the benchmark suite.

7 Experimental Setup

Table 3: Experimental configuration used for the reported results.

Parameter	Value
Benchmark instance	mk01 (10 jobs, 6 machines, 55 operations)
Population size	20
Generations	800
Random seed	42
Energy price series	Belgium, February 2022 (ENTSO-E), per-minute resolution

8 Results

8.1 Convergence

Both objectives improve steadily over generations and plateau well before the run ends, indicating the population has converged. Figures 1 and 2 show the best makespan and best energy cost, respectively, as a function of the generation number.

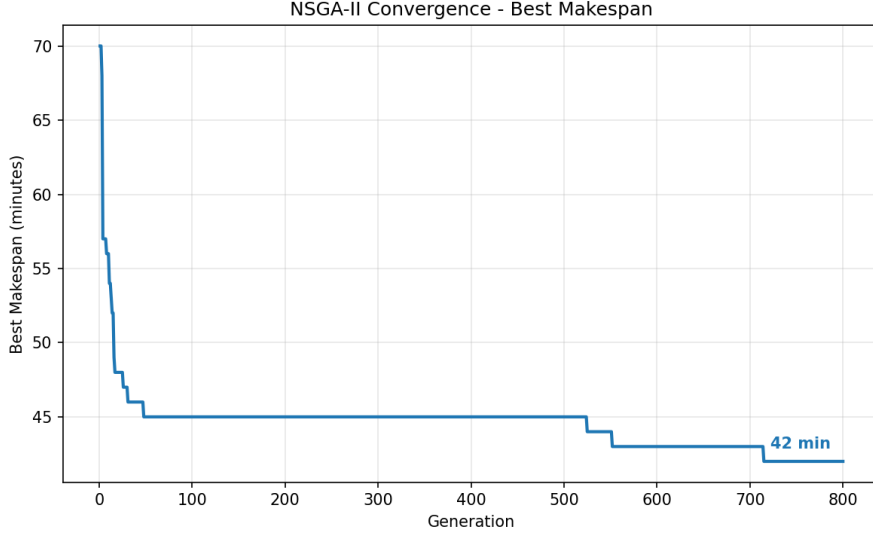


Figure 1: Best makespan per generation.

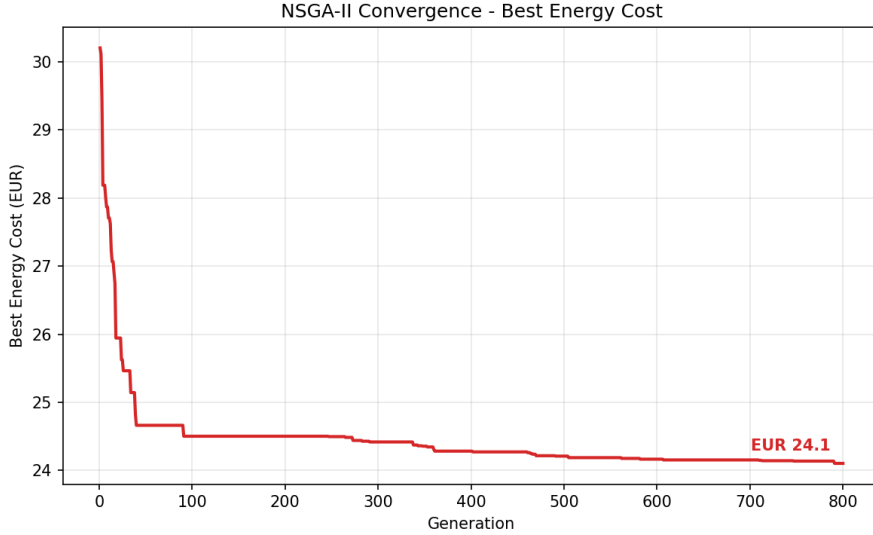


Figure 2: Best energy cost per generation.

8.2 Pareto Front

The final, deduplicated Pareto front contains 15 non-dominated solutions, trading off a 42-minute minimum-makespan schedule (higher energy cost) against an 88-minute schedule at the lowest energy cost found. Figure 3 plots the front, and Table 4 lists the full set of non-dominated objective values.

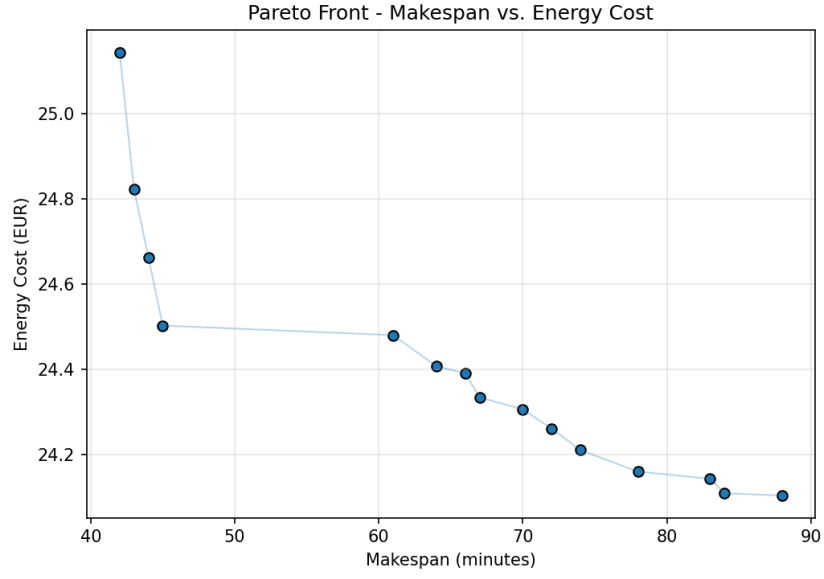


Figure 3: Pareto front: makespan vs. energy cost.

Table 4: Full Pareto front (makespan vs. energy cost), mk01, seed 42, 800 generations.

Makespan (min)	Energy Cost (EUR)
42	25.14
43	24.82
44	24.66
45	24.50
61	24.48
64	24.41
66	24.39
67	24.33
70	24.31
72	24.26
74	24.21
78	24.16
83	24.14
84	24.11
88	24.10

8.3 Schedule (Gantt Chart)

The schedule below corresponds to the minimum-makespan solution on the Pareto front (42 minutes). Each colour is a distinct job; bars are labelled with job/operation IDs where space allows.

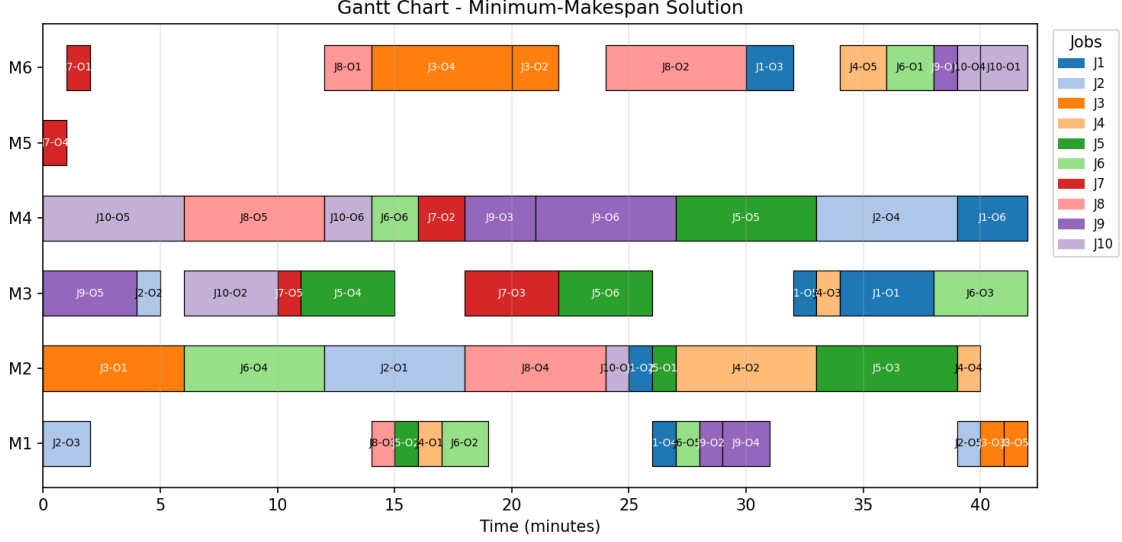


Figure 4: Gantt chart of the minimum-makespan schedule.

9 Discussion: Relationship to Burmeister et al. (2024)

This project reproduces the *concept* of a memetic NSGA-II for energy-cost-aware FJSP described in Burmeister et al. [4], not their full experimental apparatus. Specifically:

- **Benchmark coverage.** The source paper validates on all fifteen Brandimarte instances (mk01–mk15); this project includes and parses ten (mk01–mk10). The remaining five (mk11–mk15) are substantially larger and, per the source paper, pushed even an exact Gurobi solver to its memory limits; they are out of scope for a mini project run on commodity hardware.
- **No exact-solver comparison.** The source paper’s central empirical claim is a comparison of the memetic NSGA-II’s approximated front against an exact Gurobi baseline (with epsilon-constraint scalarisation) to quantify solution quality. This project does not implement or run that comparison; its Pareto fronts should be read as the algorithm’s own output, not as validated-optimal.
- **Price series.** The source paper models RTP tariffs at hourly and finer resolution over multi-day planning horizons matched to each benchmark instance’s scale. This project uses a representative one-month hourly Belgian price series, resampled to per-minute resolution.
- **Local search operator.** This project’s local search is a simplified greedy neighbourhood search over machine reassignment and sequencing swaps; the source paper’s operator design and parameterisation is more extensive.
- **No rolling horizon.** The source paper discusses extending the method to a rolling-horizon setting that reacts to updated price forecasts; this is out of scope here.

10 Limitations and Future Work

- **Computational cost of local search.** The local-search step re-evaluates a full neighbourhood every generation ($O(n_{\text{ops}})$ machine moves + $O(n_{\text{ops}}^2)$ sequencing swaps per offspring), which limits scalability to the largest benchmark instances (mk08–mk10, 20 jobs) without further optimisation.
- **No exact baseline.** Adding an exact or near-exact solver comparison (e.g. via a MILP solver on smaller instances) would let solution quality be quantified rather than only visualised.
- **Static price horizon.** The energy price series is fixed for the duration of a run; extending to a rolling-horizon setting that incorporates updated price forecasts, as discussed in Burmeister et al. [4], is a natural next step.

11 Conclusion

This project implements a memetic NSGA-II for the energy-cost-aware Flexible Job-Shop Scheduling Problem, combining real Brandimarte benchmark instances with a real electricity price series to produce genuine, non-trivial makespan/energy-cost trade-offs rather than results based on synthetic problem instances or assumed energy rates. The approach and its motivation follow Burmeister et al. [4]; this document has aimed to be explicit about what is reproduced and what is simplified, in the interest of academic honesty about the project’s scope.

References

- [1] S. Y. Abujarad, M. W. Mustafa, and J. J. Jamian. Recent approaches of unit commitment in the presence of intermittent renewable energy resources: A review. *Renewable and Sustainable Energy Reviews*, 70:215–223, 2017. doi: 10.1016/j.rser.2016.11.246.
- [2] Konstantin Biel, Fu Zhao, John W. Sutherland, and Christoph H. Glock. Flow shop scheduling with grid-integrated onsite wind power using stochastic MILP. *International Journal of Production Research*, 56(6):2076–2098, 2018. doi: 10.1080/00207543.2017.1351638.
- [3] Paolo Brandimarte. Routing and scheduling in a flexible job shop by tabu search. *Annals of Operations Research*, 41(3):157–183, 1993. doi: 10.1007/BF02023073.
- [4] Sascha C. Burmeister, Daniela Guericke, and Guido Schryen. A memetic NSGA-II for the multi-objective flexible job shop scheduling problem with real-time energy tariffs. *Flexible Services and Manufacturing Journal*, 36:1530–1570, 2024. doi: 10.1007/s10696-023-09517-7.
- [5] Daniele Carlucci, Paolo Renna, and Sergio Materi. A job-scheduling decision-making model for sustainable production planning with power constraint. *IEEE Transactions on Engineering Management*, 2021. doi: 10.1109/TEM.2021.3103108.
- [6] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T. Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197, 2002. doi: 10.1109/4235.996017.
- [7] Francesco D’Ettorre, Marzieh Banaei, Razgar Ebrahimi, et al. Exploiting demand-side flexibility: State-of-the-art, open issues and social perspective. *Renewable and Sustainable Energy Reviews*, 165:112605, 2022. doi: 10.1016/j.rser.2022.112605.
- [8] ENTSO-E Transparency Platform. Day-ahead electricity prices, belgium bidding zone, february 2022. <https://transparency.entsoe.eu/>, 2022.

- [9] Oktoviano Gandhi, Carlos D. Rodríguez-Gallegos, and Dipti Srinivasan. Review of optimization of power dispatch in renewable energy system. In *2016 IEEE Innovative Smart Grid Technologies – Asia (ISGT-Asia)*, pages 250–257, 2016. doi: 10.1109/ISGT-Asia.2016.7796394.
- [10] X. Gong, T. De Pessemier, W. Joseph, and L. Martens. An energy-cost-aware scheduling methodology for sustainable manufacturing. *Procedia CIRP*, 29:185–190, 2015. doi: 10.1016/j.procir.2015.01.041.
- [11] S. Kemmoe, D. Lamy, and N. Tchernev. A job-shop with an energy threshold issue considering operations with consumption peaks. *IFAC-PapersOnLine*, 48(3):788–793, 2015. doi: 10.1016/j.ifacol.2015.06.179.
- [12] Pablo Moscato and Carlos Cotta. A gentle introduction to memetic algorithms. In *Handbook of Metaheuristics*, pages 105–144. Springer, 2003. doi: 10.1007/0-306-48056-5_5.
- [13] SchedulingLab. fjsp-instances: Machine-readable brandimarte (1993) fjsp benchmark files in the standard hurink .fjs text format. <https://github.com/SchedulingLab/fjsp-instances>, n.d.
- [14] F. Shrouf, J. Ordieres-Meré, A. García-Sánchez, and M. Ortega-Mier. Optimizing the production scheduling of a single machine to minimize total energy consumption costs. *Journal of Cleaner Production*, 67:197–207, 2014. doi: 10.1016/j.jclepro.2013.12.024.
- [15] H. Zhang, F. Zhao, K. Fang, and J. W. Sutherland. Energy-conscious flow shop scheduling under time-of-use electricity tariffs. *CIRP Annals*, 63(1):37–40, 2014. doi: 10.1016/j.cirp.2014.03.011.